

JARVIS

System Architecture Overview

Architecture complète du système — composants, flux de données, décisions techniques

Version : 1.0 · Statut : Validé · Dossier : 04_System_Architecture / SAO-001
Déploiement : Local (Docker Compose) → Cloud · LLM : Claude API + Router multi-LLM

Décision	Choix retenu	Justification
Déploiement MVP	PC local via Docker Compose	Rapidité de dev, zéro coût infra, cloud-ready dès le départ
Cible commerciale	Cloud (Railway / Fly.io / AWS)	Même image Docker — zéro refactoring pour passer en prod
Utilisateurs	Multi-utilisateurs dès le MVP	Isolation données, auth robuste, préparation commercialisation
LLM principal	Claude API (Anthropic)	Raisonnement supérieur, context window large, SDK mature
LLM secondaires	Gemini Flash + Groq (gratuits)	Tâches légères sans coût — router intelligent
LLM fallback	Ollama local	Mode offline, zéro dépendance cloud
Backend	Python / FastAPI	Écosystème IA natif, LangChain, pgvector, async
Frontend	Next.js + TailwindCSS	SSR, performance, DX excellente
Base de données	PostgreSQL + pgvector	Relationnel + vectoriel dans un seul système
Cache / Sessions	Redis	Mémoire court terme ultra-rapide
Automatisations	n8n (self-hosted)	400+ intégrations, workflows visuels, HTTP webhooks
Domotique	Home Assistant (V2)	Exclu MVP — connecteur préparé architecturalement
Intégrations MVP	Gmail + Google Calendar	Drive et Notion en V2
Voix MVP	STT Web Speech API (input)	TTS et mode Iron Man en V2
Mémoire	RAG 3 niveaux (court/épisodique/travail)	Performance, coûts maîtrisés, pertinence contextuelle

1. Résumé exécutif

JARVIS est architecturé comme un système en couches découplées : une interface utilisateur web (Next.js), un backend d'orchestration IA (FastAPI/Python), un routeur LLM intelligent, un système de mémoire à trois niveaux (Redis + PostgreSQL + pgvector), et une couche d'intégrations externes (Google, n8n). Chaque couche est indépendante, testable et remplaçable. L'ensemble tourne en local via Docker Compose pour le MVP et se déploie en cloud sans modification de code.

Ce document décrit l'architecture complète du système JARVIS. Il constitue la référence technique pour toutes les décisions de développement, de priorisation et d'intégration. Toute modification structurelle doit être documentée ici avant d'être implémentée.

2. Vue d'ensemble de l'architecture

2.1 Principes architecturaux non négociables

- Découplage strict : chaque composant communique via des interfaces définies — jamais par appels directs entre modules non adjacents.
- Cloud-ready by design : zéro hardcoding d'IP, de chemins locaux ou de secrets. Tout passe par variables d'environnement dès le premier commit.
- Modularité du LLM : le système ne parle jamais directement à Claude ou Gemini — il passe par un LLM Router qui abstrait le choix du modèle.
- Mémoire à trois niveaux : court terme (session), épisodique (faits persistants), travail (tâches, agenda, état du jour).
- Sécurité by design : isolation des données par utilisateur, chiffrement de la mémoire personnelle, auth JWT + OAuth dès le MVP.
- Observabilité : chaque appel LLM est loggé avec son coût en tokens, sa latence et son résultat — indispensable pour maîtriser les coûts.

2.2 Les 6 couches du système

L'architecture JARVIS est organisée en 6 couches verticales qui se superposent du client jusqu'aux services externes. Chaque couche a une responsabilité unique et communique uniquement avec la couche adjacente.

Couche	Responsabilité	Technologie	Scope
L1 — Client	Interface utilisateur web, voix STT, dashboard	Next.js + TailwindCSS	MVP
L2 — API Gateway	Authentification, routing des requêtes, rate limiting	FastAPI + JWT	MVP
L3 —	Gestion du contexte, LLM Router,	LangChain / LangGraph	MVP

Couche	Responsabilité	Technologie	Scope
Orchestrateur IA	agents, RAG		
L4 — Mémoire	Court terme (Redis), épisodique (pgvector), travail (PostgreSQL)	Redis + PostgreSQL	MVP
L5 — Intégrations	Gmail, Calendar, n8n workflows, Home Assistant (V2)	APIs REST + n8n	MVP/V2
L6 — LLM	Claude (principal), Gemini Flash, Groq, Ollama (fallback)	LLM Router	MVP

3. Composants détaillés

3.1 Frontend — Interface utilisateur

Next.js 14 · TailwindCSS · Web Speech API · TypeScript

Le frontend est une Single Page Application construite avec Next.js en mode App Router. Il constitue le seul point de contact de l'utilisateur avec le système. Il ne contient aucune logique métier — il délègue tout au backend via des appels API REST.

Responsabilités

- Interface de chat principal avec streaming des réponses (Server-Sent Events)
- Capture vocale via Web Speech API — transcription côté navigateur, texte envoyé au backend
- Dashboard : tâches en cours, mémoires récentes, Morning Briefing du jour
- Gestion de session utilisateur (JWT stocké en httpOnly cookie)
- Affichage des contextes de projets — switch de contexte depuis l'interface
- Mode sombre / clair, personnalité JARVIS configurable

Ce que le frontend ne fait PAS

- Aucun appel direct aux APIs LLM — tout passe par le backend
- Aucune logique mémoire — le frontend ne sait pas ce que JARVIS se rappelle
- Aucun stockage de données sensibles côté client

Stack Frontend MVP

Framework : Next.js 14 (App Router + Server Components)
Style : TailwindCSS + shadcn/ui
Langage : TypeScript
Voix : Web Speech API (natif navigateur — zéro dépendance externe)
Streaming : Server-Sent Events pour les réponses LLM en temps réel
Auth client : JWT httpOnly cookie — jamais en localStorage
Déploiement : Container Docker exposé sur port 3000

3.2 API Gateway — Backend FastAPI

Python 3.12 · FastAPI · JWT · OAuth2 · Docker

Le backend est le cerveau du système. Il reçoit toutes les requêtes du frontend, vérifie l'authentification, orchestre les appels au LLM Router et à la mémoire, et retourne les réponses. C'est ici que réside toute la logique métier de JARVIS.

Responsabilités

- Authentification : validation des JWT, gestion des sessions, OAuth Google
- Routing des requêtes : chaque endpoint a une responsabilité unique et documentée

- Orchestration : coordination entre LLM Router, mémoire, et intégrations externes
- Streaming : passage des réponses LLM au frontend via SSE sans buffering
- Logging : chaque appel LLM tracé avec tokens consommés, latence, coût estimé
- Rate limiting : protection contre les abus, quotas par utilisateur

Structure des endpoints MVP

Endpoint	Méthode	Description	Auth
/api/auth/login	POST	Authentification email/password — retourne JWT	Public
/api/auth/google	GET	Initiation OAuth Google (Gmail + Calendar)	Public
/api/chat/message	POST	Envoi d'un message — déclenche orchestration complète	JWT
/api/chat/stream	GET	Stream SSE de la réponse LLM en cours	JWT
/api/memory/facts	GET/POST	Lecture et écriture des faits mémorisés	JWT
/api/memory/search	POST	Recherche sémantique dans la mémoire vectorielle	JWT
/api/tasks	GET/POST/PATCH	CRUD complet des tâches	JWT
/api/briefing/morning	GET	Génération du Morning Briefing du jour	JWT
/api/projects	GET/POST	Gestion des contextes de projets	JWT
/api/integrations/google	GET	Status et données Gmail + Calendar	JWT
/api/voice/transcribe	POST	Réception du texte transcrit et traitement	JWT
/api/admin/usage	GET	Logs de consommation tokens par utilisateur	Admin

3.3 LLM Router — Couche d'abstraction des modèles

Claude API · Gemini Flash · Groq · Ollama

Le LLM Router est le composant le plus stratégique de l'architecture. C'est lui qui décide quel modèle appeler pour chaque requête. Cette abstraction est fondamentale : si Anthropic change sa tarification demain, ou si un nouveau modèle devient supérieur, on change le routeur — pas le reste du code.

Logique de routing

Type de tâche	Modèle choisi	Raison	Coût estimé
Raisonnement complexe, mémoire, briefing	Claude Sonnet 4.6	Meilleur raisonnement long terme	~0.01\$/req
Questions simples, calculs, météo, sport	Gemini 2.5 Flash	Gratuit, rapide, suffisant	Gratuit

Type de tâche	Modèle choisi	Raison	Coût estimé
Réponses ultra-rapides, commandes système	Groq (Llama 3.3)	Latence < 200ms, gratuit	Gratuit
Mode offline, fallback sans internet	Ollama local (Mistral)	Zéro dépendance cloud	Gratuit
Traitement par lot, résumés longs	Claude API Batch	50% moins cher que temps réel	~0.005\$/req

Règles de fallback automatique

- Si Claude API retourne une erreur 429 (rate limit) → bascule sur Gemini Flash
- Si Gemini Flash est indisponible → bascule sur Groq
- Si aucune connexion internet → bascule sur Ollama local
- Si Ollama n'est pas installé → réponse d'erreur explicite à l'utilisateur

Token budget par requête

- Context window max injectée par requête : 8 000 tokens (mémoire + prompt + historique)
- Réponse max : 2 000 tokens — au-delà, JARVIS résume et propose de continuer
- Alerte automatique si un utilisateur dépasse 50 000 tokens/jour
- Rapport mensuel de consommation accessible depuis le dashboard

3.4 Système de mémoire — Architecture à 3 niveaux

Redis · PostgreSQL · pgvector · RAG

La mémoire est le composant le plus différenciant de JARVIS. Elle est organisée en trois niveaux distincts avec des rôles, des durées de vie et des technologies différentes. C'est cette architecture qui permet à JARVIS de "se souvenir" sans exploser les coûts LLM.

Niveau	Type	Technologie	Durée de vie	Contenu
Niveau 1	Court terme	Redis (in-memory)	Session active	30 derniers messages, contexte de la conversation en cours
Niveau 2	Épisodique	PostgreSQL + pgvector	Permanent	Faits sur l'utilisateur, préférences, événements importants, résumés de projets
Niveau 3	Travail	PostgreSQL (relationnel)	Quotidien/ hebdo	Tâches, agenda du jour, état des projets, Morning Briefing

Fonctionnement du RAG (Retrieval Augmented Generation)

À chaque message entrant, l'orchestrateur exécute la séquence suivante avant d'appeler le LLM :

- Étape 1 — Vectorisation : la question de l'utilisateur est convertie en vecteur d'embedding (text-embedding-3-small d'OpenAI ou nomic-embed local)
- Étape 2 — Recherche sémantique : pgvector recherche les 5 faits les plus proches dans la mémoire épisodique
- Étape 3 — Récupération contextuelle : les tâches du jour et l'état des projets actifs sont récupérés depuis PostgreSQL

- Étape 4 — Compression : si le contexte total dépasse 6 000 tokens, un résumé automatique est généré pour les éléments les moins pertinents
- Étape 5 — Injection : mémoire court terme + faits pertinents + contexte de travail sont injectés dans le system prompt
- Étape 6 — Post-traitement : les nouveaux faits mentionnés dans la réponse sont extraits et sauvegardés en mémoire épisodique

3.5 Couche d'intégrations externes

Gmail · Google Calendar · n8n · Home Assistant (V2)

Les intégrations externes sont gérées par un service dédié qui fait le pont entre JARVIS et les APIs tierces. Ce service est appelé uniquement par l'orchestrateur — jamais directement par le frontend ou d'autres composants.

Intégrations MVP

Service	Fonctionnalités MVP	Authentification	Statut
Gmail	Lecture des derniers emails, filtrage par expéditeur/sujet, résumé IA	OAuth 2.0 Google	MVP
Google Calendar	Lecture agenda du jour/semaine, création d'événements, rappels	OAuth 2.0 Google	MVP
Web Search (SerpAPI)	Recherche web temps réel pour les questions factuelles	API Key	MVP
n8n (self-hosted)	Orchestration de workflows, webhooks entrants et sortants	Token interne	MVP
Spotify	Contrôle lecture/pause/titre/volume via API Spotify	OAuth 2.0 Spotify	MVP
YouTube	Lancement de vidéos par titre/artiste via YouTube Data API	API Key Google	MVP

Intégrations V2 planifiées

- Home Assistant : connecteur préparé architecturalement — endpoint /api/integrations/ha réservé
- Notion : lecture et création de pages, sync base de données
- GitHub : issues, pull requests, status des repositories
- Slack : envoi et lecture de messages dans les canaux définis
- WhatsApp Business API : envoi de messages sur commande
- Google Drive : lecture, création et organisation de fichiers

4. Infrastructure & Déploiement

4.1 Architecture locale MVP — Docker Compose

PC local · Docker · Port mapping · Volumes persistants

Pour le MVP, JARVIS tourne intégralement sur le PC de développement via Docker Compose. Un seul fichier docker-compose.yml lance l'ensemble de la stack en une commande. Les données sont persistées dans des volumes Docker pour survivre aux redémarrages.

Services Docker Compose — MVP

Service	Image	Port	Volume	Rôle
jarvis-frontend	node:20-alpine (build)	3000	—	Interface Next.js
jarvis-backend	python:3.12-slim (build)	8000	—	API FastAPI + orchestrateur
jarvis-postgres	postgres:16-alpine	5432	postgres_data	Base relationnelle + pgvector
jarvis-redis	redis:7-alpine	6379	redis_data	Cache sessions + mémoire court terme
jarvis-n8n	n8nio/n8n:latest	5678	n8n_data	Workflows et automatisations
jarvis-ollama	ollama/ollama:latest	11434	ollama_models	LLM local fallback

Commande de démarrage

Lancement du stack complet JARVIS

```
$ git clone https://github.com/[user]/jarvis
$ cd jarvis
$ cp .env.example .env      # Renseigner les clés API
$ docker-compose up -d      # Lance tous les services
$ open http://localhost:3000 # Interface JARVIS
```

Arrêt : docker-compose down
Mise à jour : docker-compose pull && docker-compose up -d

4.2 Architecture cloud cible — Commercialisation

Railway / Fly.io / AWS · CI/CD · HTTPS · Monitoring

Quand JARVIS passe en mode commercial, la même image Docker se déploie sur une infrastructure cloud. Le code ne change pas — seules les variables d'environnement et la configuration réseau changent. C'est l'avantage d'avoir respecté les principes cloud-ready dès le MVP.

Composant	Solution recommandée	Alternative	Raison du choix
Hébergement backend	Railway (simple) ou Fly.io	AWS ECS / GCP Cloud Run	Déploiement Git-push, pricing prévisible
Base de données	Neon (PostgreSQL serverless)	Supabase / RDS Aurora	pgvector natif, scaling automatique
Redis cloud	Upstash Redis	Redis Cloud / ElastiCache	Serverless, pay-per-use, faible coût
CDN / Frontend	Vercel (Next.js natif)	Netlify / Cloudflare Pages	Optimisé pour Next.js, edge functions
Monitoring	Sentry + Posthog	Datadog / New Relic	Open source friendly, pricing startup
CI/CD	GitHub Actions	GitLab CI / CircleCI	Intégration native GitHub, gratuit
Secrets	Railway Variables / AWS SSM	HashiCorp Vault	Gestion centralisée des clés API
n8n cloud	n8n Cloud ou self-hosted VPS	Make / Zapier	Même workflows, migration transparente

5. Flux de données — Séquences clés

5.1 Flux principal — Message utilisateur → Réponse JARVIS

Ce flux décrit le chemin complet d'un message depuis la saisie utilisateur jusqu'à la réponse affichée. Il représente 80% du volume de requêtes du système.

Éta pe	Composant	Action	Durée cible
1	Frontend (browser)	Utilisateur envoie un message (texte ou voix transcrite)	< 10ms
2	API Gateway	Validation JWT, identification utilisateur, rate limit check	< 20ms
3	Orchestrateur	Récupération mémoire court terme (Redis) — 30 derniers messages	< 30ms
4	Orchestrateur → pgvector	Vectorisation de la question + recherche sémantique (top 5 faits)	< 100ms
5	Orchestrateur → PostgreSQL	Récupération tâches du jour + contexte projets actifs	< 30ms
6	Orchestrateur	Construction du prompt final avec contexte complet (compression si > 6K tokens)	< 50ms
7	LLM Router	Choix du modèle selon type de tâche + appel API LLM	~1000-3000ms
8	API Gateway → Frontend	Streaming de la réponse via SSE token par token	Continu
9	Orchestrateur (async)	Extraction et sauvegarde des nouveaux faits en mémoire épisodique	Arrière-plan
10	Frontend	Affichage de la réponse complète + mise à jour UI	< 50ms

Temps total cible (hors LLM) : < 300ms. Temps de première réponse (streaming) : < 1500ms.

5.2 Flux Morning Briefing

Le Morning Briefing est déclenché automatiquement au premier accès de la journée ou manuellement depuis le dashboard. Il agrège des données de plusieurs sources en une synthèse personnalisée.

- Gmail → Récupération des emails non lus des dernières 12h, filtrage des importants
- Google Calendar → Événements du jour et du lendemain
- PostgreSQL → Tâches en retard + tâches prioritaires du jour
- SerpAPI → Météo du jour (lieu configuré dans le profil utilisateur)
- PostgreSQL → Résumé des projets actifs avec dernière mise à jour
- LLM Router (Claude) → Synthèse narrative personnalisée de toutes les données
- Redis → Sauvegarde du briefing du jour pour éviter une double génération

5.3 Flux d'authentification Google (OAuth 2.0)

- L'utilisateur clique "Connecter Google" dans les paramètres
- Le frontend redirige vers `/api/auth/google` — le backend initie le flow OAuth
- Google affiche la page de consentement (Gmail + Calendar scopes)
- Google redirige vers `/api/auth/google/callback` avec un code d'autorisation
- Le backend échange le code contre `access_token` + `refresh_token`
- Les tokens sont chiffrés et stockés en base (jamais exposés au frontend)
- Le backend rafraîchit automatiquement l'`access_token` avant expiration

6. Schéma de base de données — PostgreSQL

PostgreSQL est le système de persistance central de JARVIS. Il héberge à la fois les données relationnelles classiques (utilisateurs, tâches, projets) et les vecteurs d'embedding pour la mémoire sémantique via l'extension pgvector.

6.1 Tables principales

Table	Colonnes principales	Rôle	Relations
users	id, email, name, hashed_password, personality_mode, created_at	Comptes utilisateurs	→ memories, tasks, projects
memories	id, user_id, content, embedding (vector 1536), memory_type, importance_score, created_at	Mémoire épisodique vectorielle	→ users
conversations	id, user_id, messages (JSONB), project_id, tokens_used, created_at	Historique des conversations	→ users, projects
tasks	id, user_id, title, description, priority, status, due_date, project_id	Gestion des tâches	→ users, projects
projects	id, user_id, name, description, context_summary, is_active	Contextes de projets	→ users
briefings	id, user_id, content, date, weather_data, calendar_data, email_summary	Morning Briefings générés	→ users
integrations	id, user_id, provider, access_token_enc, refresh_token_enc, scopes, expires_at	Tokens OAuth chiffrés	→ users
llm_logs	id, user_id, model, prompt_tokens, completion_tokens, cost_usd, latency_ms, created_at	Audit consommation LLM	→ users

6.2 Index critiques pour les performances

- memories.embedding → Index HNSW pgvector (recherche sémantique ANN rapide)
- memories.user_id + memory_type → Index composite (filtrage par utilisateur et type)
- tasks.user_id + status + due_date → Index composite (requêtes dashboard)
- conversations.user_id + created_at → Index pour récupération historique
- llm_logs.user_id + created_at → Index pour rapports de consommation

7. Sécurité & Protection des données

La mémoire JARVIS contient des informations personnelles sensibles : préférences, projets, emails résumés, habitudes de travail. La sécurité n'est pas une option — elle est architecturée dès le MVP et non ajoutée après coup.

Vecteur d'attaque	Mesure de protection	Implémentation	Priorité
Accès non autorisé	JWT + refresh tokens rotation	FastAPI JWT middleware, expiry 1h/7j	Critique
Vol de données mémoire	Chiffrement AES-256 des données sensibles	Fernet (cryptography lib Python)	Critique
Injection de prompt	Sanitisation des inputs, system prompt protégé	Validation Pydantic + prefix injection guard	Critique
Fuite de tokens OAuth	Tokens chiffrés en base, jamais exposés au client	Chiffrement Fernet + httpOnly cookies	Critique
Cross-user data leak	Isolation stricte user_id sur toutes les requêtes	RLS PostgreSQL + middleware de vérification	Critique
Abus API (DDoS)	Rate limiting par utilisateur et par IP	Redis + FastAPI slowapi	Haute
Secrets exposés	Variables d'environnement uniquement, jamais dans le code	.env + Docker secrets, Git ignore strict	Critique
Logs sensibles	Anonymisation des contenus dans les logs LLM	Hashing des contenus, logs de métadonnées only	Haute

8. Registre des décisions d'architecture (ADR)

Chaque décision architecturale majeure est documentée ici avec sa justification et les alternatives écartées. Ce registre est la mémoire du projet — il évite de prendre deux fois la même mauvaise décision.

ADR-001 — Python/FastAPI comme backend principal

Décision : Python 3.12 + FastAPI pour l'intégralité du backend.

Justification : L'écosystème IA est natif Python : LangChain, pgvector, Whisper, tous les SDKs Anthropic/OpenAI. Utiliser Node.js aurait nécessité de wrapper des libs Python ou d'utiliser des portages incomplets. Pour un projet centré IA, c'est rédhibitoire.

Alternatives écartées : Node.js/Express (écarté — écosystème IA pauvre), Go (écarté — trop bas niveau pour prototyper rapidement des agents IA), Django (écarté — trop verbeux pour une API REST pure).

ADR-002 — LLM Router — couche d'abstraction obligatoire

Décision : Aucun composant du système n'appelle directement l'API Claude ou Gemini. Tout passe par le LLM Router.

Justification : Les APIs LLM changent rapidement (tarification, modèles, disponibilité). Une abstraction permet de changer de modèle ou de fournisseur en modifiant un seul fichier de configuration. Sans cette couche, chaque changement de provider nécessite une refonte du code.

Alternatives écartées : Appels directs aux APIs (écarté — couplage fort, migration impossible), LiteLLM (envisagé — bonne option mais surcharge pour le MVP, on implémente notre propre router simple).

ADR-003 — PostgreSQL + pgvector — une seule base de données

Décision : Un seul système de base de données pour le relationnel ET le vectoriel.

Justification : Ajouter une base vectorielle dédiée (Pinecone, Weaviate, Chroma) en plus de PostgreSQL double la complexité opérationnelle, les coûts et les points de défaillance. pgvector offre des performances suffisantes pour un MVP et même au-delà jusqu'à plusieurs millions de vecteurs.

Alternatives écartées : Pinecone (écarté — coût + dépendance cloud), Chroma (écarté — pas production-ready), Weaviate (écarté — complexité opérationnelle), Redis Vector (écarté — persistance moins fiable).

ADR-004 — Voix STT navigateur uniquement au MVP

Décision : Web Speech API côté navigateur pour la transcription. Le texte transcrit est envoyé au backend. Pas de TTS dans le MVP.

Justification : Whisper (OpenAI) ajoute 500-1500ms de latence et un coût par minute d'audio. Web Speech API est gratuit, natif, et transcrit en temps réel. Le TTS (synthèse vocale) ajoute un autre niveau de latence et de coût qui n'est pas justifié avant validation du concept core.

Alternatives écartées : Whisper API (écarté MVP — latence + coût), ElevenLabs TTS (écarté MVP — coût + complexité), Google Speech-to-Text (écarté MVP — OAuth supplémentaire).

ADR-005 — Docker Compose pour le déploiement local

Décision : L'intégralité du stack tourne dans Docker Compose avec des volumes persistants.

Justification : Docker garantit la reproductibilité de l'environnement entre machines et facilite la migration vers le cloud. Un développeur qui clone le repo a JARVIS fonctionnel en moins de 10 minutes. Les volumes persistent les données entre redémarrages.

Alternatives écartées : *Installation directe sur le système (écarté — non reproductible, dépendances conflictuelles), Kubernetes (écarté — sur-ingénierie pour le MVP), Virtual machine (écarté — trop lourd).*

ADR-006 — Gemini Flash + Groq comme LLM secondaires gratuits

Décision : Les tâches simples (météo, calculs, réponses courtes) sont routées vers Gemini 2.5 Flash ou Groq — tous deux gratuits.

Justification : Claude API coûte de l'argent. Gemini 2.5 Flash reste gratuit pour les requêtes légères (limité à ~100-1500 req/jour selon quotas Google). Groq est ultra-rapide et gratuit sur les modèles open source. Un router intelligent permet de réduire les coûts de 60-70% sans dégrader l'expérience.

Alternatives écartées : *Claude pour tout (écarté — coûts trop élevés à l'échelle), OpenAI GPT-4o-mini (envisagé — payant, moins pertinent que Groq gratuit pour les tâches simples).*

9. Roadmap technique — Phases de développement

Phase	Durée	Composants à construire	Critère de done
0 — Cadra ge	2 sem.	Documentation, architecture, choix stack, structure du repo	Tous les documents fondateurs produits et validés
1a — Fonda tions	2 sem.	Docker Compose, PostgreSQL + pgvector, Redis, FastAPI skeleton, auth JWT	Stack qui démarre en une commande, auth fonctionnelle
1b — LLM Core	2 sem.	LLM Router, intégration Claude API, mémoire court terme Redis, chat basique	Conversation avec Claude depuis l'interface, mémoire de session
1c — Mémoi re	2 sem.	Mémoire épisodique pgvector, RAG pipeline, extraction de faits post-réponse	JARVIS se souvient des faits entre sessions
1d — Featur es MVP	2 sem.	Tâches, projets, Morning Briefing, voix STT, Gmail + Calendar, dashboard	Toutes les features MVP fonctionnelles et testées
1e — Polish MVP	1 sem.	Tests, correction de bugs, optimisation performances, documentation utilisateur	MVP stable, latence < 2s, prêt pour premiers utilisateurs
2 — Autom atisati ons	6-10 sem.	n8n workflows, TTS voix, Notion, Slack, domotique HA, mode Iron Man	Intégrations V2 fonctionnelles
3 — Agent s	Long terme	Multi-agents LangGraph, proactivité, API publique, mobile app	Plateforme commerciale scalable

9.1 Ordre de développement Phase 1 — Justification

L'ordre des sous-phases de la Phase 1 est délibéré et non négociable. Chaque sous-phase construit sur la précédente.

- Les fondations (1a) doivent être solides avant tout le reste — une base bancale fait tout s'effondrer
- Le LLM Core (1b) valide l'architecture avant d'investir dans la mémoire complexe
- La mémoire (1c) est construite sur un LLM Core validé — pas l'inverse
- Les features MVP (1d) utilisent tous les composants précédents — impossible avant
- Le polish (1e) est la semaine la plus importante — une demo instable ne convainc personne

10. Structure du repository GitHub

La structure du repository reflète l'architecture du système. Chaque dossier a une responsabilité unique.

Structure du repository jarvis/

```
jarvis/
├── frontend/           # Next.js 14 application
│   ├── app/           # App Router pages et layouts
│   ├── components/    # Composants UI réutilisables
│   └── lib/           # Utilities, hooks, API client
├── backend/           # FastAPI application
│   ├── api/           # Routes FastAPI (endpoints)
│   ├── core/          # Config, sécurité, DB connections
│   ├── services/      # Logique métier (orchestrateur, mémoire, intégrations)
│   │   ├── llm_router.py # LLM Router — choix du modèle
│   │   ├── memory.py    # Gestion mémoire 3 niveaux
│   │   ├── orchestrator.py # Orchestrateur principal
│   │   └── integrations/ # Gmail, Calendar, Spotify...
│   ├── models/        # Modèles SQLAlchemy + Pydantic schemas
│   └── tests/         # Tests unitaires et d'intégration
├── n8n/               # Workflows n8n exportés (JSON)
├── docs/              # Documentation technique
├── docker-compose.yml  # Stack complète locale
├── docker-compose.prod.yml # Override pour production
├── .env.example        # Template variables d'environnement
└── Makefile           # Commandes utiles (make start, make test...)
```